# Assignment -06

Ram Prosad Sarkar.

Topic

Modes of operation and Res-Block Diagram and Java Implementation

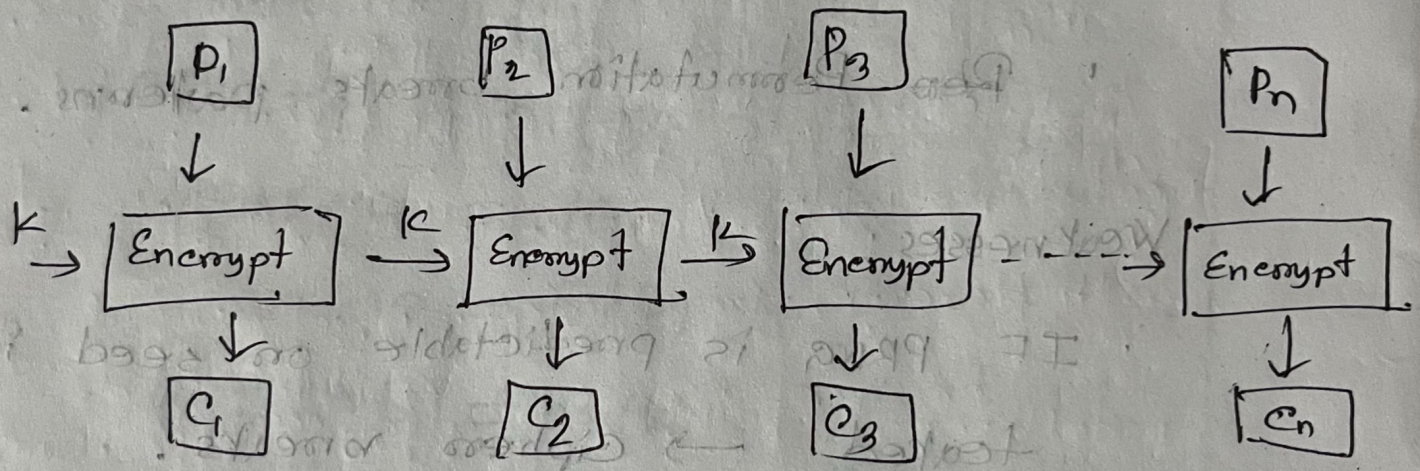
## Modes of operation

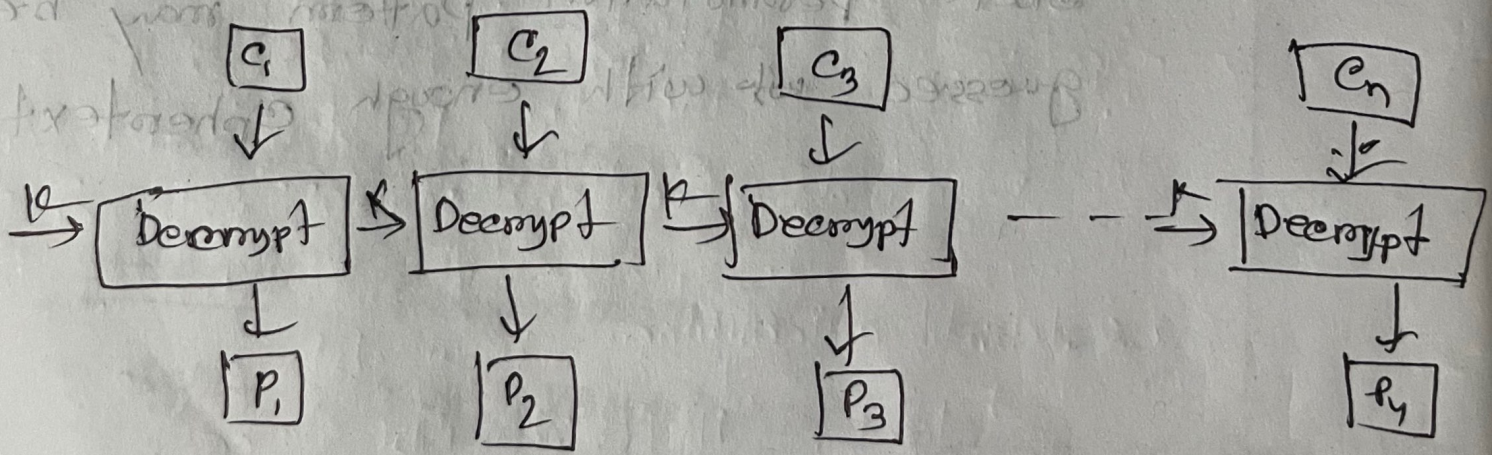
- ECB: Each block is encrypted independently, not secure for patterns.
- CBC: XORs each plaintext block with privious ~~chip~~ ciphertext block before encryption.
- CFB: converts block cipher into a self-Synchronizing stream cipher.
- OFB: Turns block cipher into a synchronous ~~syst~~ stream cipher.
- CTR: Uses a counter that gets encrypted and XORed with plaintext fast and parallelizable.

Prasad

ECB (Electric Codebook): In an electric code book each block of bits of plaintext is encoded independently with the same key.

Block diagram:



Rupprasad

ECB ~~Java~~ implementation with Java

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpace;
import java.util.Base64;

Public class ECBMode {
    public static SecretKey generateAESkey () {
        throw Exception {
            KeyGenerator keyGen = KeyGenerator.get
                Instance ("AES");
            keyGen.init(128);
            return keyGen.generateKey ();
        }
    public static string encrypt (string plain text,
        SecretKey key) throw Exception {
        Cipher cipher = Cipher.getInstance ("AES/ECB/
            PKCS5padding");
        Cipher.init (cipher.ENCRYPT.MODE.secret);
```

Rampraasad

```
byte[] encryptedBytes = cipher.doFinal(
    plaintext.getBytes());
    return Base64.getEncoder().encode-
    toString(encryptBytes);}

public static string decrypt(string encryptedText,
    secretKey secretKey) throws Exception{
    Cipher cipher = Cipher.getInstance("AES/ECB/
                                    pkcs5 padding")
    Cipher.init(cipher.DECRYPT_MODE, SecretKey);
    byte[] decryptedBytes = cipher.doFinal(Base64.
    getDecoder)
            decode(encryptedText));
    return new string(decryptedBytes);
    public static void main(string[] args){
        try{ secretKey = generateAESkey();
            String original = "Hello cyber world!";
            String encrypted = encrypt(originalKey);
```

Rampraasad

```
String decrypted = decrypt(encrypted, key);
System.out.println("original text" +original);
System.out.println("encrypted Text:" +encrypted);
System.out.println("Decrypted Text:" +decrypted);
}
catch (Exception e){
    e.printStackTrace();
}}
```

output :

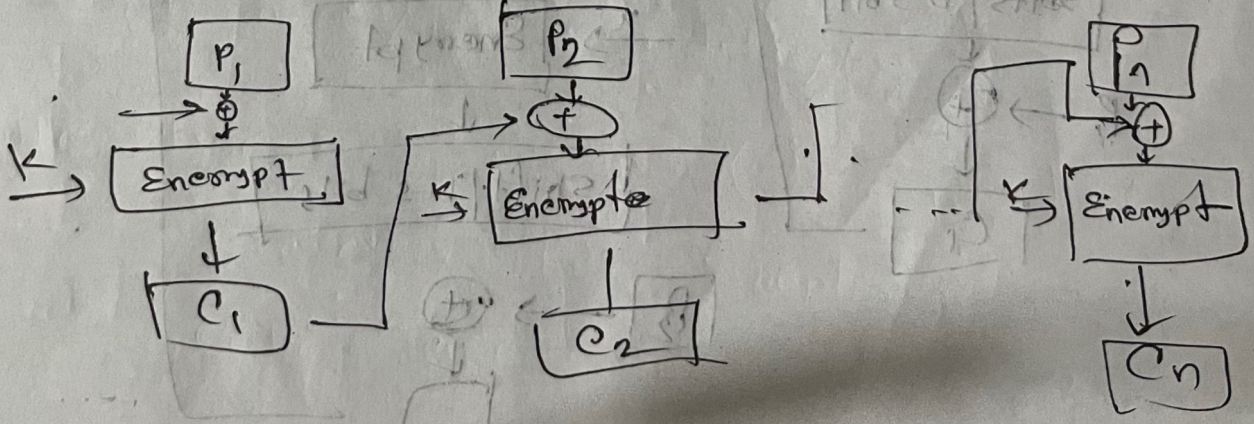
Original Text : Hello cyber world!

Encrypted Text : fRQH YBP 3A BC RPAMATNAR@P

Decrypted Text : Hello cyber world!

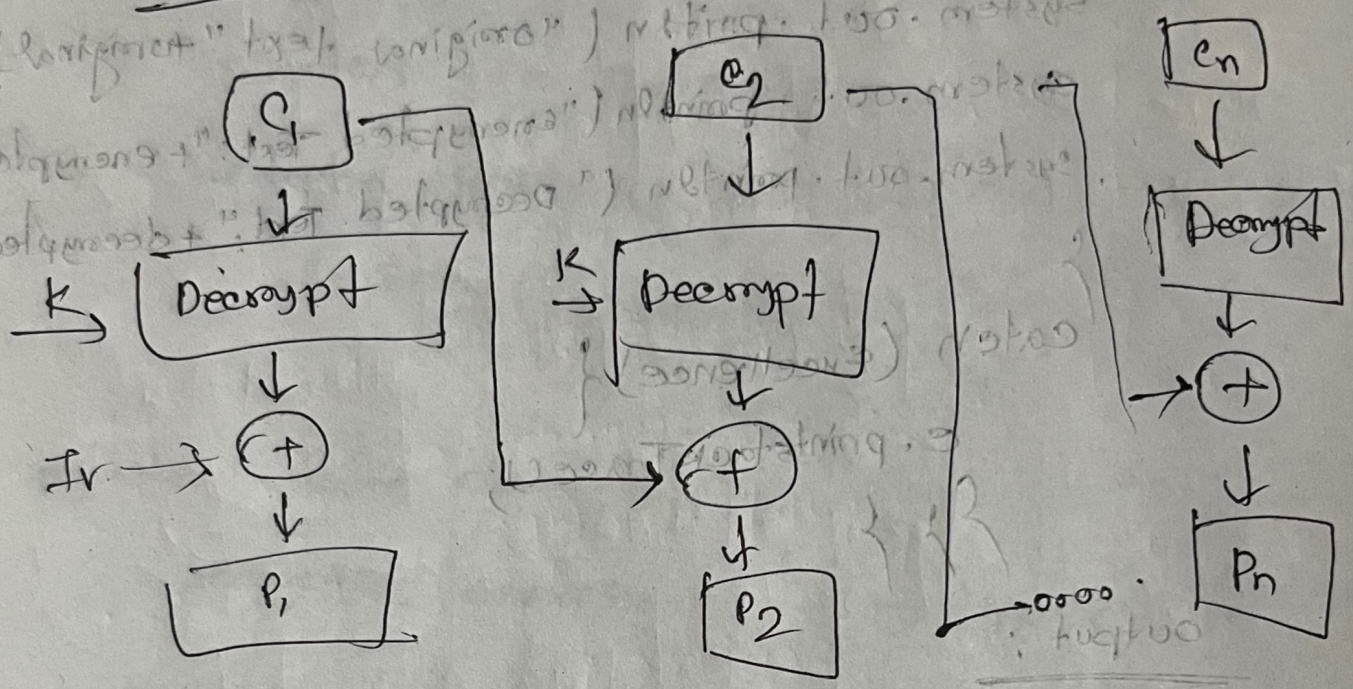
## CBC (Cipher Block ~~Chaining~~ Chaining)

Encryption

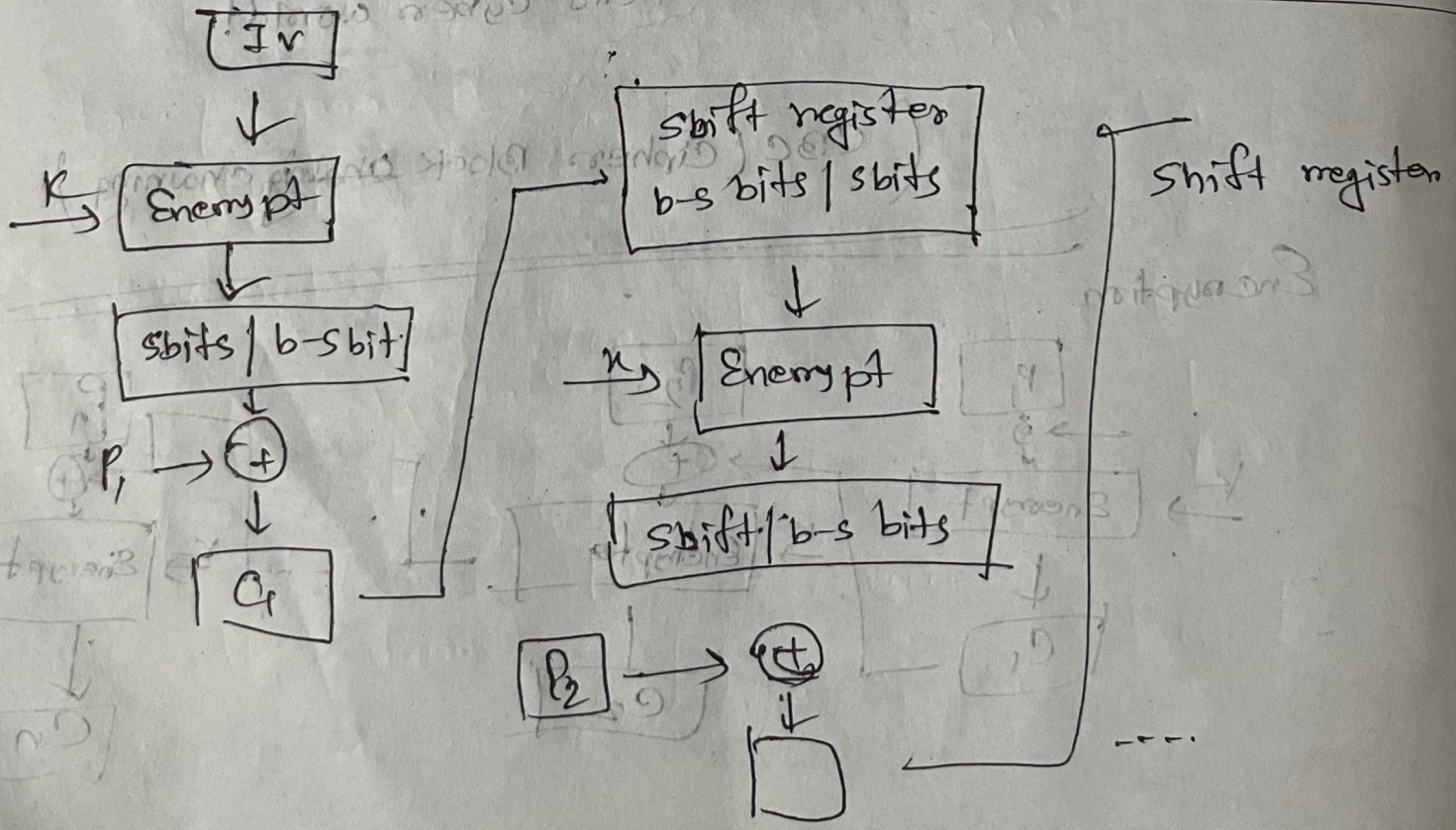


Ramprasad

## Decryption

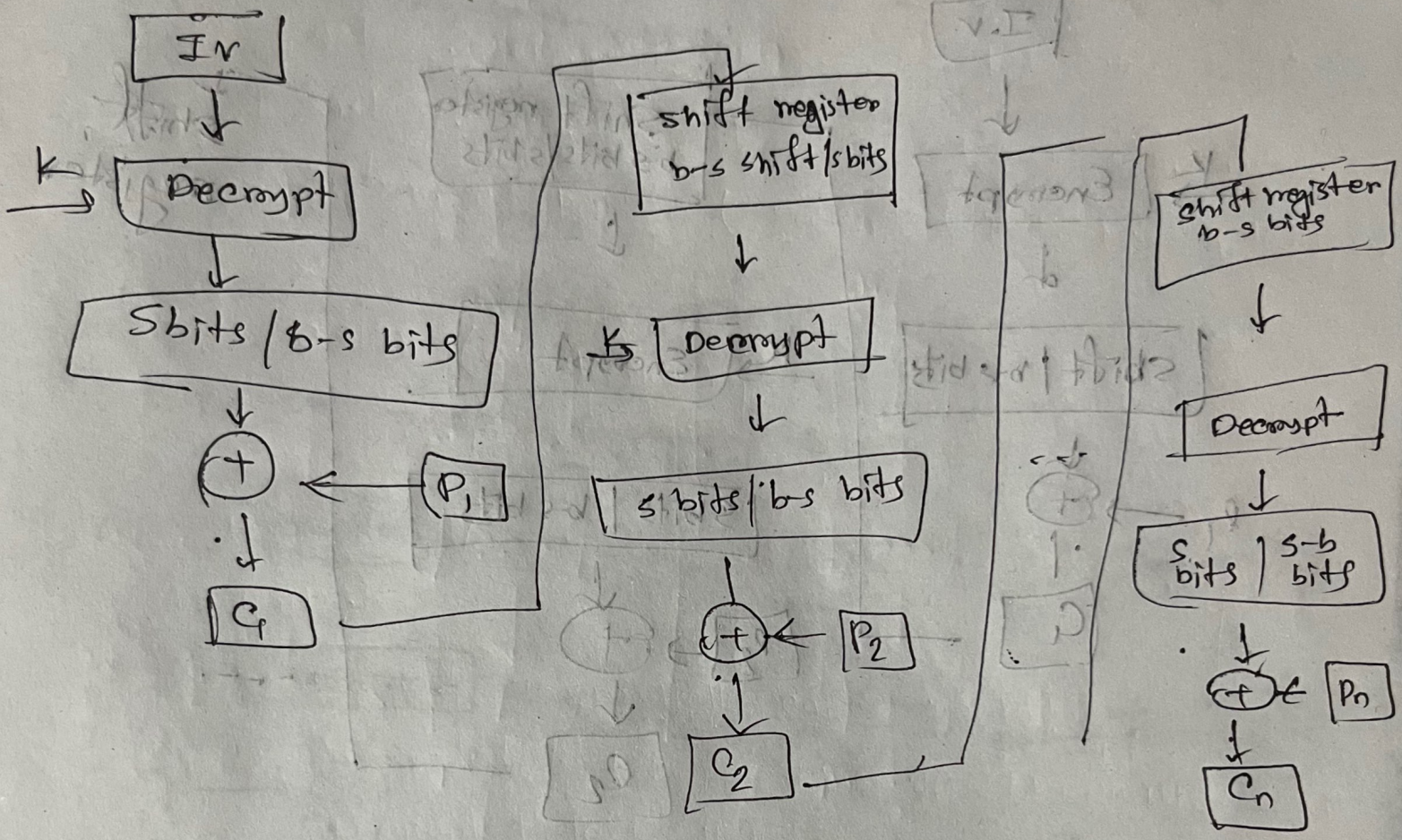



## CFB (Cipher Feedback mode)



Ramprasad

Decryption



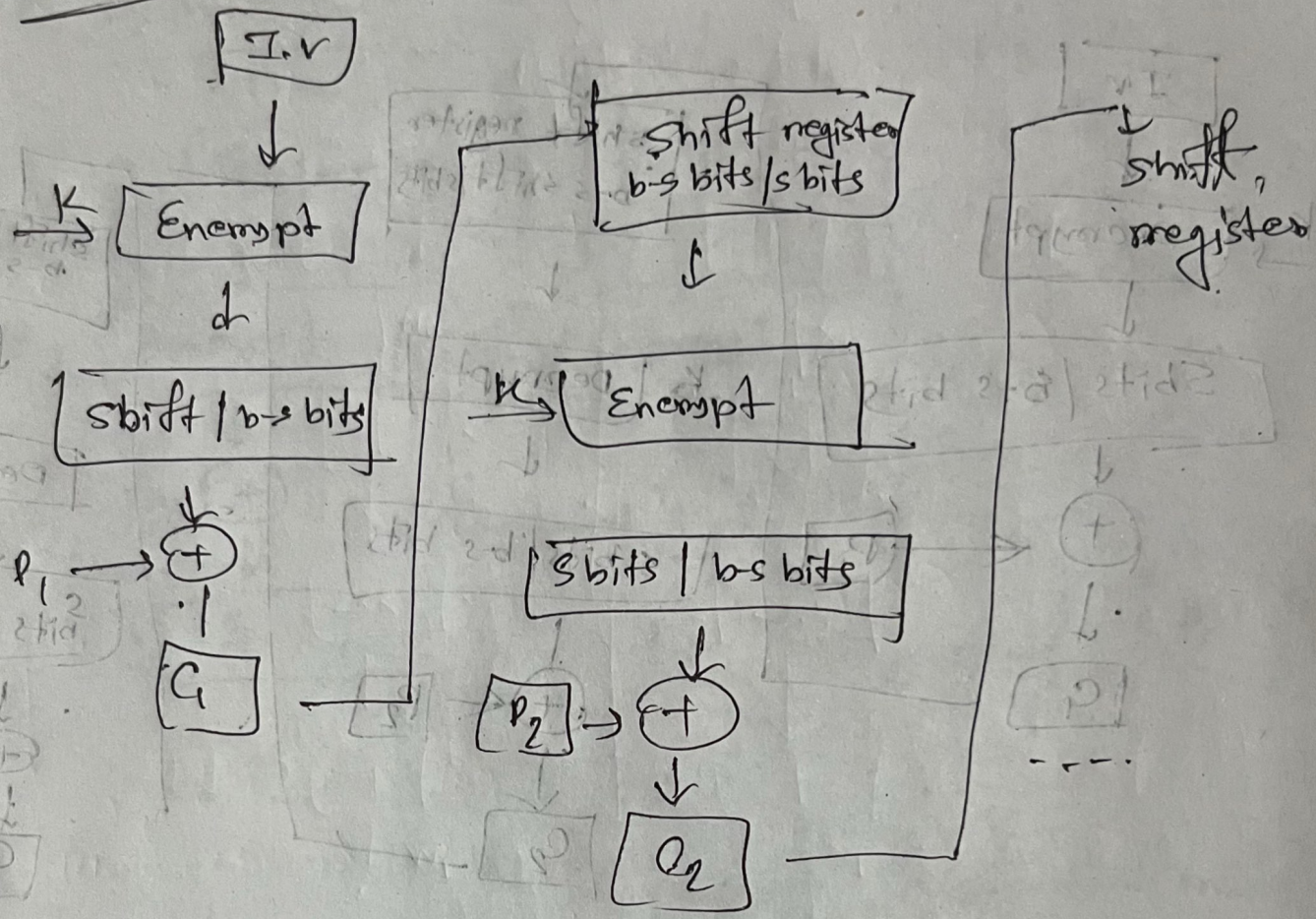

CFB

It encrypts the previous ciphertext and XORs it with the current plaintext blocks.

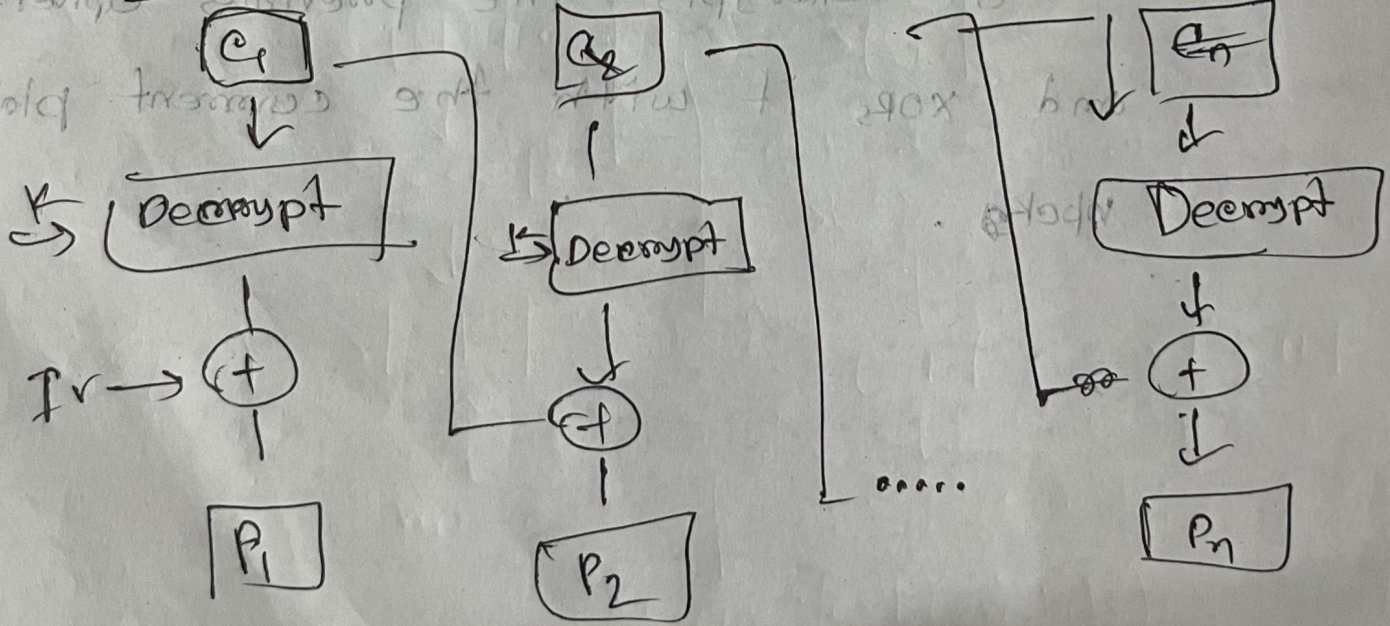
Ramprasad

# OFB

## Encryption




## Decryption

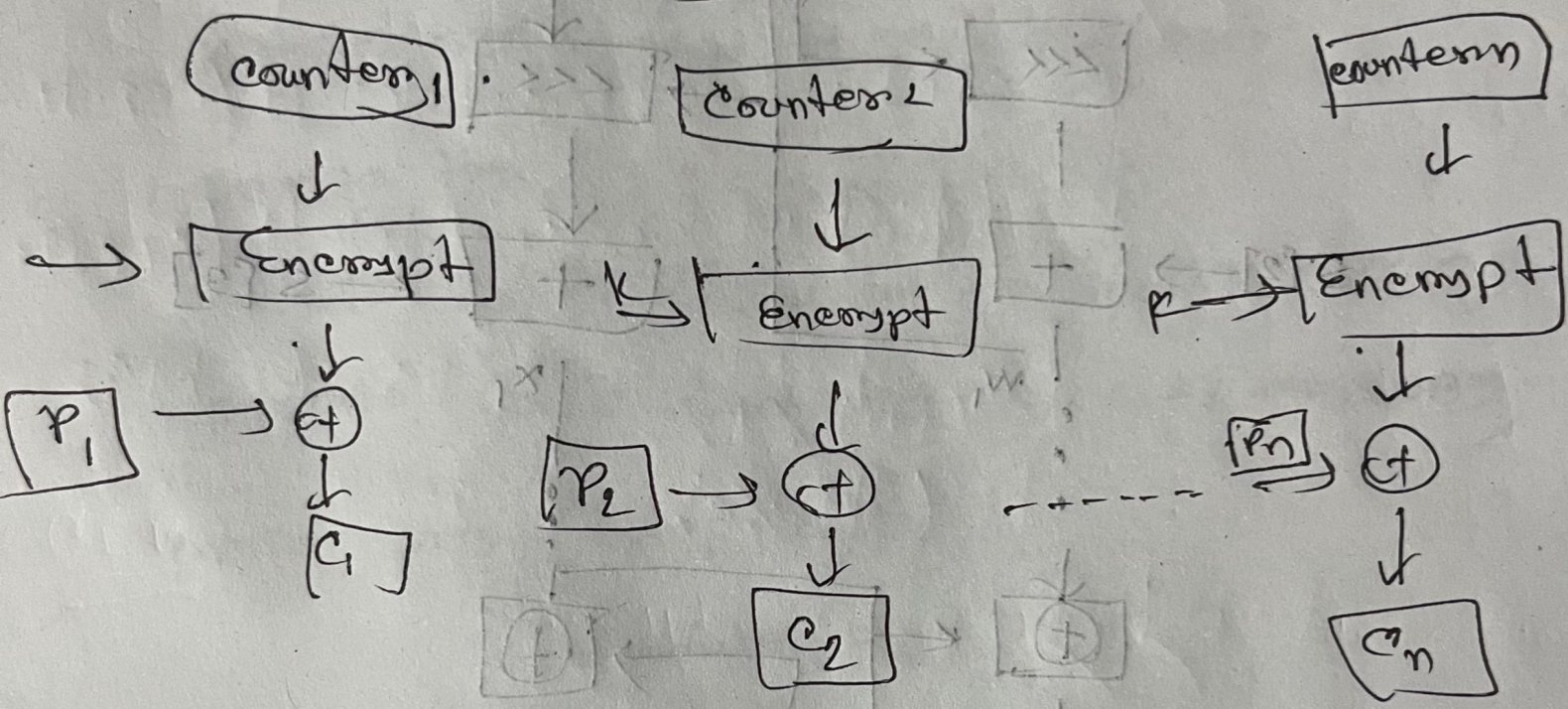


~~Output feedback~~

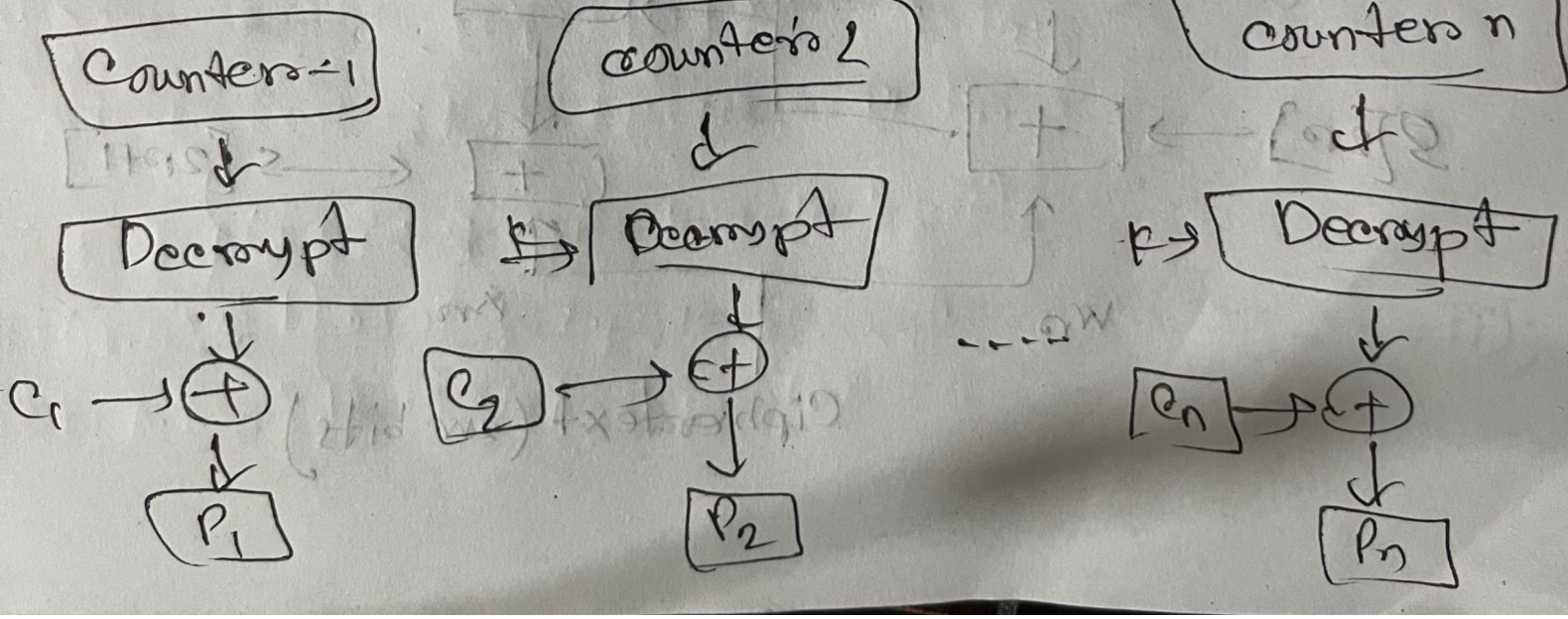
CTR

Counter mode

Counter mode Encryption

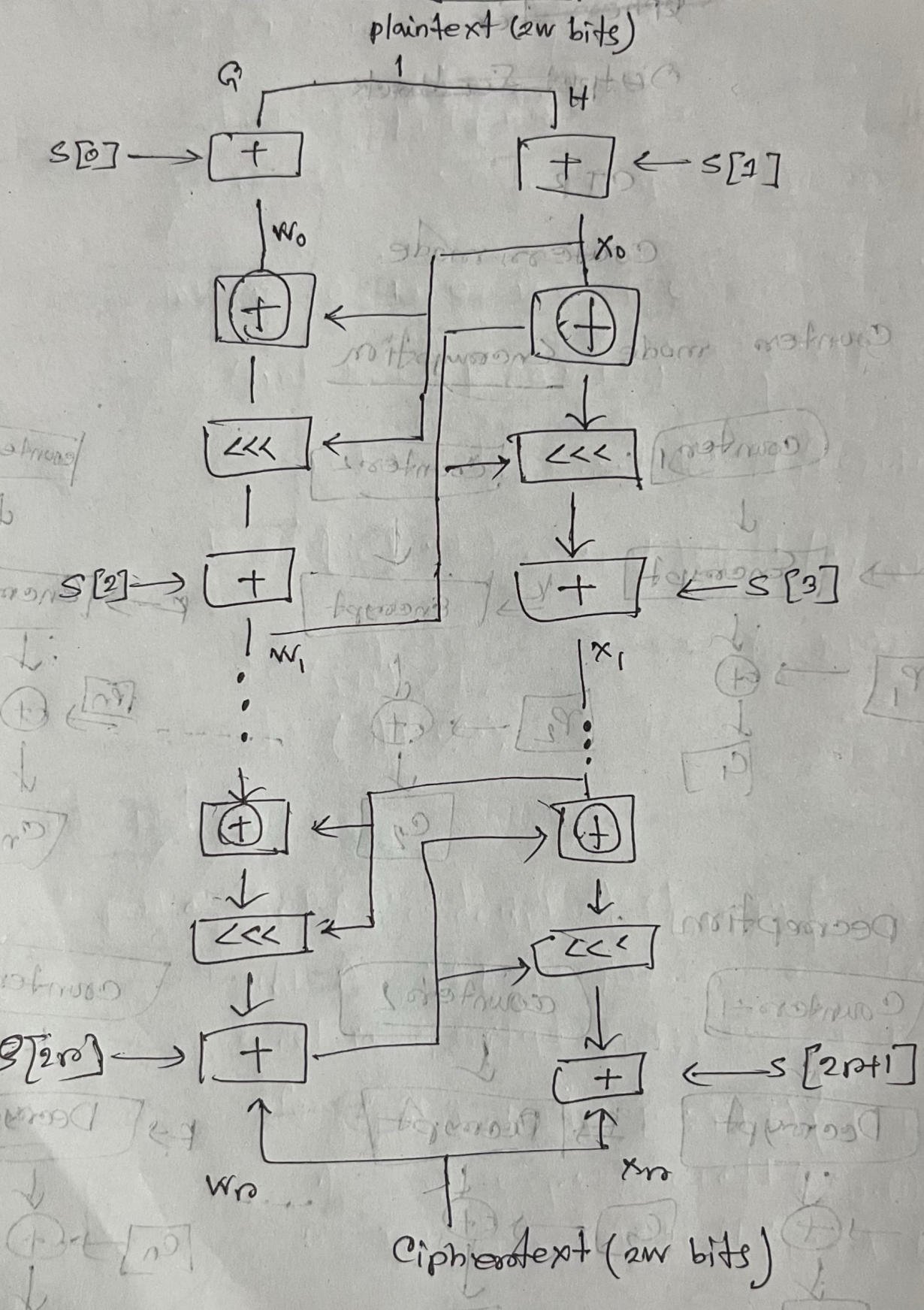



Decryption



Ramprasad

# RC5 - Block Diagram



Rompprosad

Java Implementation

```
Public class RC5 {

    private static final int WORD_SIZE = 38;
    private static final int R = 12;
    private static final int B = 8;
    private static final int C = B/4;
    private static final int T = 2*(R+1);
    private int [] S = new int[T];
    private static final int P = 0xB7E15163;
    private static final int Q = 0x9E3779B9;

    public RC5 (byte[] key) {
        keySchedule (key);
    }

    private void keySchedule (byte[] key) {
        int[] L = new int[C];
        for (int i=0; i<B; i++) {
            L[i/4] = (L[i/4] << B) + (key[i] & 0xFF);
```

Rompasad

```
S[0] = P
for (int i=1; i<T; i++){
    S[i] = S[i-1] + Q;
}
int A=0; B=0; i=0; j=0;
for(int k=0; k<3*T; k++){
    A = S[i] = Integer.rotateleft((S[i]+A+B),3);
    B = L[j] = Integer.rotateleft((L[j]+A+B),
                                  (A+B));
    i = (i+1)%T;
    j = (j+1)%C;
}
public int[] encrypt(int[] pt){
    int A = pt[0]+S[0];
    int B = pt[1]+S[1];
    for (int i=1; i<=R; i++){
        A = Integer.rotateleft(A^B, B)+S[2*i];
        B = Integer.rotateleft(B^A, A)+S[2*i];
    }
```

Rompasad

```
return new int[] {A,B};
}

public int[] decrypt (int[] ct){
    int B = ct[1];
    int A = ct[0];
    for(int i=R; i>=1; i--){
        B = Integer.rotateRight(B-S[2*i+1],A)^A;
        A = Integer.rotateRight(A-S[2*i],B)^B;
    }
    A -= S[0];
    B -= S[1];
    return new int[] {A,B};
}

public static void main (String[] args){
    byte[] key = "password".getBytes();
    RC5 rc5 = new RC5(key);
    int[] pt = {0x12345978, 0xabcdef0};
    int[] ct = rc5.encrypt(pt);
```

```
System.out.printf("Encrypted: %08x%08x\n",
                  ct[0], ct[1]);
int[] dt = rsa.decrypt(ct);
System.out.print("Decrypted: %08x
                  %08x\n", dt[0], dt[1]);
```

Sample output:

Encrypted: 7f03d8e2 1432ba29

Decrypted: 12345678 9abcdef2

Explanation:

- The input plaintext is: 0x12345678 0x9abcdef0
- The encrypted ciphertext looks random
- After decryption we get back the exact original plaintext.